Yelp Review Intelligence: NLP Pipeline v3

Feature engineering, sentiment/emotion analysis, and an interactive dashboard built on the Yelp Open Dataset (8.6M reviews) — with a validated, reproducible single-machine pipeline that refuses to ship fake data.

8.6M Reviews

Streamed from raw JSON; never fully loaded into memory

60+ Features

Text, sentiment, emotion, affect, entities, transformers

Validated Output

Pipeline validates throughout its composite files

Iteration 3

Scaled down for readily-applicable industry case studies

Project Scope & Case Study Industries

Dataset & Stack

Source: Yelp Open Dataset — 8.6M reviews, streamed from raw JSON; never fully loaded into memory

Stack: Python, pandas, spaCy, VADER, NRCLex, NRC-VAD Lexicon v2.1, HuggingFace Transformers (optional), Streamlit

Output per review: 60+ engineered feature columns spanning text, linguistics, sentiment, emotion, affect dimensions, named entities, and optional transformer scores

Industry Case Studies

Industry 1 — Mexican Restaurants: Chipotle locations; 15,000-review fixed-seed sample drawn via category substring-match on each business's Yelp categories field

Industry 2 — Hair Salons: Great Clips locations; 15,000-review fixed-seed sample using the same mechanism

Sampling logic: `numpy.default_rng(seed=222)` ensures byte-identical reproducibility across runs without redistributing Yelp data

Three Iterations: What Changed and Why

Iteration 1 — MSBA 502

Working sentiment/emotion analysis + regression models on a 15k random sample. Introduced HuggingFace DistilBERT sentiment and j-hartmann emotion classifier. Baseline $R^2 = 0.25$.

1

2

Iteration 2 — MSBA 503

Deployed full AWS/Spark pipeline at 8.6M-review scale. Sampled downstream, to prepare two potential industry analyses via Streamlit Dashboard.

New Applications

3

Decided to re-work the pipeline to sample first, then apply compute-heavy NLP processes, for targeted business applications versus big-data analysis from the jump.

4

Iteration 3 — This Repo

A pipeline that runs without distributed processing, with transformer models optional, and a baseline of Vader lexicon sentiment scoring, for small-scale use and versatility.

Pipeline Architecture: Two-Script Design

Script 1 — make_samples.py

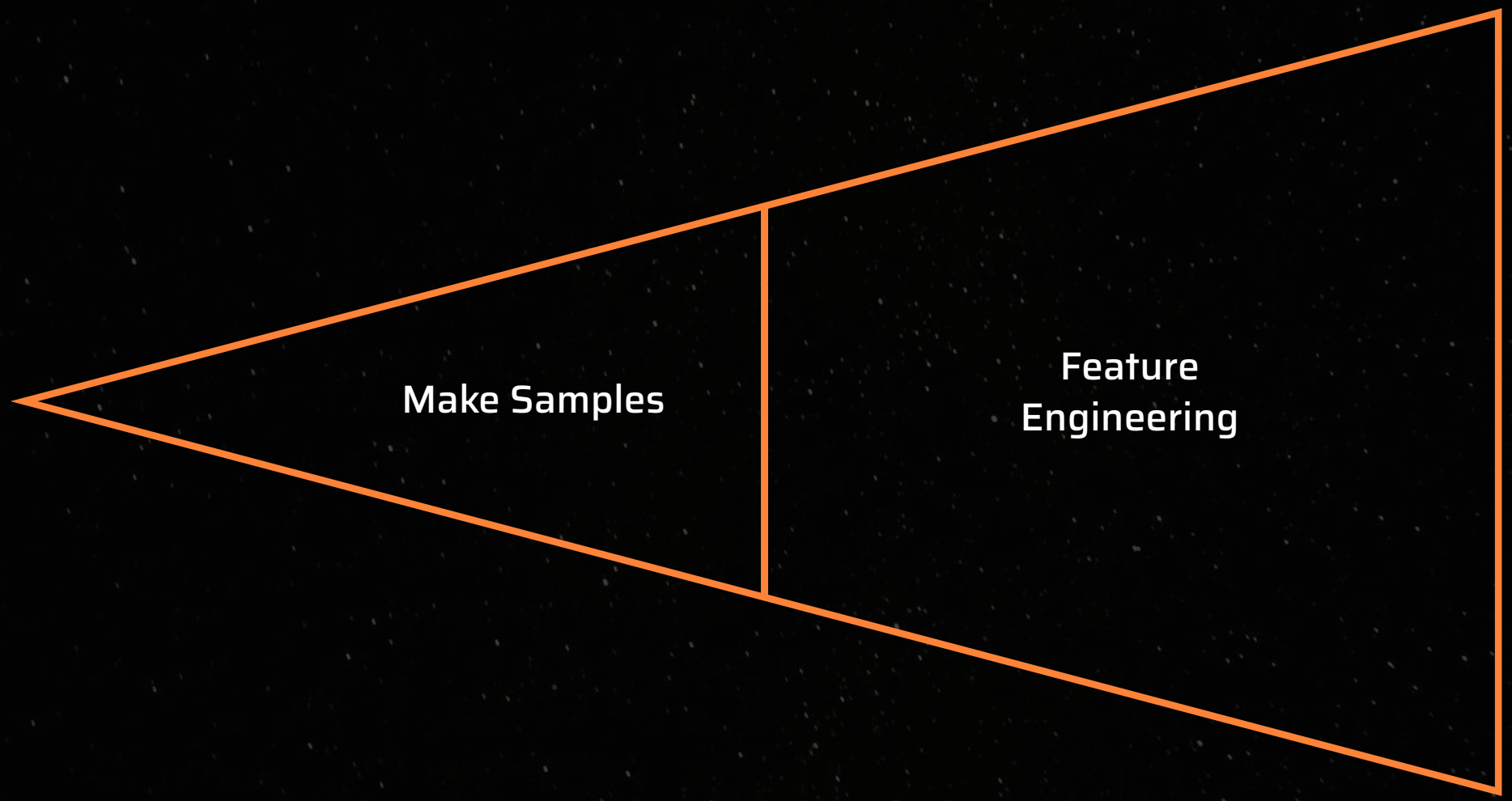
Yelp raw JSON → reproducible industry sample CSVs. Handles business indexing via category substring-match, optional user enrichment, and two-pass streaming to avoid loading 8.6M reviews into RAM simultaneously.

Separation of concerns: Sampling is a one-time reproducibility step. Run it once; the output CSVs are the stable inputs to script 2.

Script 2 — real_feature_engineering.py

Sample CSVs → validated, feature-rich output CSVs. All 60+ columns computed from raw text on every run. The `GENERATED_COLUMNS` constant validates that variables are unique.

Outputs: `{name}_REAL.csv` per input + an appended entry in `pipeline_run_log.json`. Failed validation raises `AssertionError` and writes nothing.



Import-safe design: No side effects at import time — `init_models()` must be called explicitly, keeping notebook usage clean and predictable.

make_samples.py: Reproducible Streaming Sampler

Input Files

- `yelp_academic_dataset_business.json`
- `yelp_academic_dataset_review.json`
- `yelp_academic_dataset_user.json` (optional)

Key Packages

- `numpy` — seeded RNG for reproducibility
- `pandas` — CSV output
- `json` — streaming line-by-line parse
- `pathlib`, `argparse`

Processing Logic

Business indexing: Single pass over business JSON; case-insensitive substring match on the `categories` field; first matching industry wins per business.

Two-pass review streaming: Pass 1 counts matching reviews per industry (low memory). Pass 2 collects only the pre-chosen row indices — the full 8.6M is never held in RAM.

Reproducibility: `numpy.default_rng(seed=222)` selects indices before Pass 2. Identical inputs + identical seed = byte-identical output CSVs.

User enrichment: `--no-users` skips the user JSON for faster runs; when included, adds `user_name`, `user_stars_avg`, `user_review_count`.

CLI example: `python make_samples.py --json-dir path/to/Yelp-JSON --industry "Mexican:chipotle" "Hair Salons:hair" --n 15000 --seed 222 --outdir ../data/raw`

real_feature_engineering.py: Pipeline Entry Points

init_models()

Loads spaCy `en_core_web_sm` (NER + sentencizer), VADER, NRCLex instance, and NRC-VAD Lexicon v2.1. Must be called once before any processing. Parser, tagger, and lemmatizer are disabled for speed.

init_transformers()

Optional. Loads HuggingFace DistilBERT sentiment + j-hartmann emotion classifier. Requires `pip install transformers torch` — not in default requirements due to multi-GB torch footprint.

process_dataframe()

Iterates rows, calls all feature functions, concatenates feature columns onto the original DataFrame. Accepts any CSV with a review-text column — configurable via `--text-col` (default: `raw_review`).

validate()

Runs three sanity checks post-processing. Raises `AssertionError` and blocks file write on failure. The structural safeguard that iteration 2 lacked entirely.

GENERATED_COLUMNS

Exhaustive list of all 60+ pipeline-owned columns. Any column present in the input CSV is dropped and fully regenerated — old values are never trusted or carried forward.

Notebook-safe: `from pipeline.real_feature_engineering import init_models, process_dataframe, validate` — zero side effects at import time.

Feature Category 1: Text Cleaning & Linguistics

clean_text()

Strips URLs (`http\S+|www\.\S+`), collapses whitespace and newlines. Order matters — URL removal first prevents double spaces. Returns the **full cleaned text** as `stripped_review`, not the truncated mock from iteration 2.

spaCy Configuration

`en_core_web_sm` loaded with lemmatizer, attribute_ruler, tagger, and parser disabled — only NER and sentencizer active, minimizing per-review overhead.

Sentence boundaries: spaCy sentencizer pipe (lightweight); `sentence_count` floored at 1 to avoid division-by-zero in `avg_sentence_len`.

Key Packages

spaCy `en_core_web_sm`, `re`, `numpy`

11 Linguistic Feature Columns

- word_count
- char_count
- avg_word_len
- avg_sentence_len
- sentence_count
- num_excl
- num_ques
- num_caps
- num_at
- num_hash
- negation_count

Negation detection: 30-word lookup set covering contractions (can't, isn't, n't, etc.); counts negation tokens per review as a complement to VADER's internal handling.

Feature Category 2: VADER Sentiment (4 Columns)

What VADER Provides

VADER (Valence Aware Dictionary and sEntiment Reasoner) — rule-based lexicon tuned for social media and short informal text (Hutto & Gilbert, 2014). Handles punctuation emphasis (**!!!**), capitalization (**AMAZING**), and common slang without retraining. Fast CPU inference, no model download required.

4 Output Columns

- **vader_sentiment_score** — compound score, range [−1, 1]
- **vader_pos** — proportion of positive sentiment
- **vader_neu** — proportion of neutral sentiment
- **vader_neg** — proportion of negative sentiment

Proportional scores sum to ~1.0 per review.

Key package:
`vaderSentiment.vaderSentiment.SentimentIntensityAnalyzer`

Validation Role

VADER compound score is the **primary signal in the post-processing sanity check**. Correlation with star ratings must exceed **r = 0.30** or the pipeline refuses to write output.

Observed Correlations (vs. Star Ratings)

0.69

Chipotle r

VADER compound vs. stars

0.74

Great Clips r

VADER compound vs. stars

0.30

Min. Threshold

Pipeline blocks write below this

Feature Category 3: NRC Emotion Lexicon (18 Columns)

Lexicon

NRC Emotion Lexicon (Saif M. Mohammad, NRC Canada) — word-level binary associations across 8 emotions + positive/negative polarity. One shared `NRCLex()` instance is reused across all rows via `load_token_list(simple_tokenize(text))`; tokenizer uses `re.compile(r"[a-zA-Z']+")` for speed.

Implementation Note

dominant_emotion: Highest-proportion emotion; "neutral" if no emotion words matched. Ties broken deterministically by `EMOTION_ORDER` position — explicit, not accidental.

Key package: `nrclex.NRCLex`

18 Columns Across Three Groups

Count columns (10): Raw lexicon hit counts per review — `{emotion}_count` for all 8 emotions + `positive_count`, `negative_count`

Proportion columns (8): `{emotion}_int_avg` — each emotion's share of all emotion-bearing words in the review. Naming kept for schema compatibility with iteration 2; documented as *proportions*, not intensity averages.

Dominant emotion (1): `dominant_emotion` — argmax of `_int_avg`

8 Emotions

- anger
- fear
- joy
- sadness
- anticipation
- disgust
- surprise
- trust

Feature Category 4: NRC-VAD Affect Dimensions (4 Columns)

Lexicon

NRC Valence-Arousal-Dominance Lexicon v2.1 (Mohammad, 2018) — continuous scores in [−1, 1] for 54,801 terms. Not redistributable; downloaded locally per setup instructions. Single-word entries only (44,728 of 54,801 rows); multi-word phrases skipped — marginal gain vs. added lookup complexity at this scale.

4 Output Columns

- **Valence_avg** — pleasantness; word-level scores averaged across matched tokens
- **Arousal_avg** — activation/energy level
- **Dominance_avg** — sense of control
- **vad_matched_words** — match count for downstream filtering

Key packages: `pandas` (lexicon load + dict lookup), `numpy` (mean aggregation)

Design Decisions

Zero-match handling: Reviews with no lexicon matches return 0.0 on all three dimensions (scale midpoint = neutral). `vad_matched_words = 0` flags these rows for downstream filtering or down-weighting.

Validation Anchors (vs. Star Ratings)

0.59

Chipotle r

Valence_avg vs. stars

0.64

Great Clips r

Valence_avg vs. stars

0.20

Min. Threshold

Pipeline blocks write below this

Top predictor: Valence_avg is the strongest positive predictor of star ratings in both industries in the final regression models.

Feature Category 5: Named Entity Recognition (3 Columns)

Method

spaCy `en_core_web_sm` NER applied to the same `doc` object already created for linguistic features — text is parsed **only once per review**. The `doc` object is returned from `linguistic_features()` and passed directly to `entity_features()`.

Key Package

spaCy `en_core_web_sm` (NER component)

3 Output Columns & Entity Mapping

person_count

Maps from: `PERSON`
Captures staff name mentions — signals service-quality and staff-specific reviews

location_count

Maps from: `GPE + LOC + FAC`
Captures multi-location comparisons and neighborhood references

product_count

Maps from: `PRODUCT + ORG`
Captures brand and menu item references within a review

Efficiency note: no redundant parse — the spaCy doc object flows through the pipeline in a single pass covering both linguistic and entity feature extraction.

Feature Category 6: HuggingFace Transformers (5 Columns, Optional)

Activation & Models

Flag: `--transformers`; requires `pip install transformers torch` (not in default requirements.txt — torch is multi-GB).

Sentiment model: `distilbert-base-uncased-finetuned-sst-2-english` — contextual binary sentiment

Emotion model: `j-hartmann/emotion-english-distilroberta-base` — 7-way emotion classification

5 Output Columns

- **hf_sentiment_label** — binary 0/1
- **hf_sentiment_confidence** — model confidence score
- **hf_emotion_label** — top emotion from 7-way classifier
- **hf_emotion_confidence** — model confidence score
- **hf_computed_sentiment** — `(2 × label − 1) × confidence`, signed [−1, 1]; formula preserved from iteration 1 for cross-iteration comparability

Why Transformers Outperform Lexicons for Emotion

A lexicon matches "sad" regardless of surrounding context. A transformer correctly reads *"sad they're closing!"* in a 5-star review as **positive sentiment**. Context-awareness is the critical difference for nuanced short-form reviews.

Compute Tradeoff

~25 min per 15k reviews on CPU (iteration 1 benchmark). GPU strongly recommended. Both `hf_*` and lexicon columns coexist in output for direct A/B comparison.

Known Limitation

Truncation: Both HF pipelines use `truncation=True` — reviews exceeding model max token length are truncated.

Lexicons vs. Transformers: The Core Tradeoff

Both feature sets coexist in the same output CSV — enabling direct A/B comparison in notebooks and the dashboard. The recommended path: run the lexicon pipeline first (fast, always available), then add `--transformers` on GPU hardware for the full feature set.

Dimension	Lexicon (VADER + NRC)	Transformer (HuggingFace)
Speed	Seconds per 15k rows	~25 min per 15k on CPU
Hardware	Any CPU	GPU recommended
Accuracy (sentiment)	Good (r = 0.69–0.74 vs stars)	Higher (contextual)
Accuracy (emotion)	Context-blind	Context-aware
Interpretability	Word-level scores	Black-box confidence
Install size	Lightweight	Multi-GB (torch)

Validation System:

Core principle: mandatory `validate()` call that blocks file writes on failure. Three independent checks, all must pass.

1

word_count Integrity

Pearson correlation between pipeline's `word_count` and actual `str.split().str.len()` on raw text must exceed **r = 0.90**. Catches truncation

2

Constant Column Detection

Any numeric column with `nunique() == 1` triggers failure. `num_at` and `num_hash` are explicitly excluded — legitimately near-zero in restaurant and salon reviews.

3

Sentiment-vs-Rating Correlation

`vader_sentiment_score` vs `stars` must exceed **r = 0.30**. `Valence_avg` vs `stars` must exceed **r = 0.20**. Both must pass simultaneously.

On Failure

`AssertionError` raised with a descriptive message. No CSV written. The pipeline "refuses to save output that fails sanity checks."

Observed Results (Both Samples Pass)

VADER r = 0.69 (Chipotle) / 0.74 (Great Clips) | Valence r = 0.59 / 0.64 | word_count r > 0.99

Audit Trail: pipeline_run_log.json

Purpose

Append-only JSON log proving every output file was produced by a validated run. Any output CSV in the repo can be traced to an exact log entry with full validation metrics.

Append-Only Guarantee

Existing entries are read, new entries appended, the full array rewritten. Past runs are **never overwritten**. Legacy dict-format logs from earlier runs are migrated automatically on first write.

Reproducibility Link

Combined with fixed-seed `make_samples.py`, any log entry can be exactly reproduced: **same seed + same input + same script = byte-identical output**.

Per-Run Entry Fields

timestamp_utc

ISO timestamp of the pipeline run

input filename & rows processed

Provenance of the source CSV

output path

Full path to the written `_REAL.csv`

vad_lexicon path used

Confirms which NRC-VAD file was loaded

transformers flag (true/false)

Whether HF models were active

full validation metrics dict

All three check results stored per run; loadable as a DataFrame for history analysis

Analysis Results: Modeling on Real Features

All models trained on the validated `_REAL.csv` outputs. The VADER vs. stars integrity check yielded r = 0.71 across combined 30,000 reviews.

Metric	Value	Notes
Linear R^2 (stars prediction)	**0.61**	Refined feature set; avg error ≈ 1 star
Classification accuracy	**91%**	Positive vs. negative binary; balanced weights
F1 — positive class	**0.93**	4–5 stars
F1 — negative class	**0.85**	1–2 stars; 3-star reviews excluded
VADER vs. stars (combined)	**r = 0.71**	Integrity check threshold: 0.30
Per-industry R^2 — Hair Salons	**0.66**	Slightly stronger signal; reviews skew more positive/expressive
Per-industry R^2 — Mexican Restaurants	**0.58**	Slightly weaker signal
Top positive driver (both industries)	Valence_avg	NRC-VAD lexicon
Top negative driver (both industries)	disgust_int_avg	NRC emotion proportion

R^2 = 0.61 on a validated lexicon-only feature set substantially outperforms the iteration 1 transformer-derived benchmark of 0.25 — at a fraction of the compute cost.

Model Diagnostics:

Multicollinearity — Resolved

Adaptive VIF pruning dropped `vad_matched_words` (a data-quality indicator, not a construct). Log-transformed `word_count` linearizes the long right tail. **Max VIF after pruning = 3.3** — all remaining features retained.

Heteroscedasticity — Confirmed & Corrected

Breusch-Pagan test p ≈ 0 (expected with an ordinal target). **HC3 robust standard errors applied** for any inference — coefficients unchanged, SEs inflated ~1.4× on average.

Ordinal Target — Acknowledged

Stars are bounded [1, 5]; predictions clipped to valid range. Residuals show diagonal striping (not normally distributed). p-values/CIs on coefficients should not be taken literally; R^2 and point predictions remain valid for benchmarking.

Clustered Observations

Reviews cluster within businesses and users, mildly understating standard errors. Acceptable for prediction; a mixed-effects model is the rigorous upgrade for causal inference.

VIF Scores After Adaptive Pruning

Feature	VIF
Valence_avg	3.34
Dominance_avg	2.44
log_word_count	2.21
negation_count	2.00
vader_sentiment_score	1.83
joy_int_avg	1.67
disgust_int_avg	1.55
anger_int_avg	1.49
sadness_int_avg	1.44
Arousal_avg	1.37
trust_int_avg	1.35
fear_int_avg	1.28
num_caps	1.14
avg_word_len	1.09
num_excl	1.04

Bottom line: Models used for prediction and feature benchmarking, not causal inference. Ordinal logistic regression or gradient-boosted trees are the natural next step for a rating-scale-respecting alternative.

Streamlit Dashboard: Four Analytical Views

Navigation: Industry → State → City → Business drilldown. Run with `streamlit run dashboard/app.py`. Imports `init_models`, `process_dataframe` directly from `pipeline.real_feature_engineering` for live scoring.

Descriptive View

Distribution of star ratings, dominant emotions, sentiment scores, and linguistic features across filtered reviews. Drill from industry to individual business.

Diagnostic View

Feature correlation explorer — identify which NLP signals most strongly predict ratings for a given business or geography. Supports hypothesis generation.

Predictive View

Live what-if review scoring — user types a review, the pipeline scores it in real time (VADER + NRC + optional transformer), and returns a predicted star rating.

Prescriptive View

Actionable recommendations derived from feature patterns — e.g., high `negation_count` + low `trust_count` as a service-failure signal surfaced to the operator.

Repo Structure & Data Setup

Directory Layout

pipeline/

`make_samples.py`, `real_feature_engineering.py`, `pipeline_run_log.json`

data/

`raw/`, `processed/`, `lexicons/` — all gitignored; regenerated locally from Yelp source

notebooks/

`01_analysis.ipynb` — EDA, modeling, per-industry comparison

dashboard/

`app.py` — Streamlit application

docs/

Data dictionary, design notes

archive/

Selected artifacts from iterations 1 & 2 — the before/after story

Data Not Committed

Yelp Open Dataset: License restricts redistribution — regenerated locally per setup instructions.

NRC-VAD Lexicon v2.1: Free for research; terms prohibit redistribution — downloaded locally.

Setup (3 Commands)

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python -m spacy download en_core_web_sm
```

Optional Transformer Install

```
pip install transformers torch
```

Not in default requirements — torch is multi-GB and GPU-recommended.

Complete Feature Schema: 60+ Columns

Category	Columns	Method / Source
Text	stripped_review	URL strip + whitespace normalization
Linguistic (11)	word_count, char_count, avg_word_len, avg_sentence_len, sentence_count, num_excl, num_ques, num_caps, num_at, num_hash, negation_count	Direct counts; spaCy sentencizer
Sentiment (4)	vader_sentiment_score, vader_pos, vader_neu, vader_neg	VADER (compound in [−1, 1])
Emotion counts (10)	anger/fear/joy/sadness/anticipation/disgust/surprise/trust _count, positive_count, negative_count	NRC Emotion Lexicon
Emotion proportions (8)	anger/fear/joy/sadness/anticipation/disgust/surprise/trust _int_avg	NRC Emotion Lexicon (share of emotion-bearing words)
Dominant emotion (1)	dominant_emotion	Argmax of _int_avg; "neutral" if no match
Affect dimensions (4)	Valence_avg, Arousal_avg, Dominance_avg, vad_matched_words	NRC-VAD Lexicon v2.1
Named entities (3)	person_count, location_count, product_count	spaCy en_core_web_sm NER
Transformers (5, optional)	hf_sentiment_label, hf_sentiment_confidence, hf_computed_sentiment, hf_emotion_label, hf_emotion_confidence	DistilBERT + j-hartmann/emotion-english-distilroberta-base

Key Engineering Decisions Worth Highlighting

GENERATED_COLUMNS Constant

Exhaustive list of all pipeline-owned columns. Any present in input are dropped before processing.

Single spaCy Parse Per Review

`linguistic_features()` returns the `doc` object; `entity_features()` consumes it directly. Text is never parsed twice — a meaningful efficiency gain across 15,000-row batches.

Shared NRCLex Instance

One `NRCLex()` object reused across all rows via `load_token_list()`. Avoids repeated object initialization overhead at scale without sacrificing correctness.

Sparse Column Exclusion in Validation

`num_at` and `num_hash` are excluded from the constant-column check. Legitimately near-zero in restaurant and salon review.

*_int_avg Naming Preserved

Schema compatibility with iteration 2 maintained despite the semantic mismatch (these are proportions, not intensity averages). Documented in code and the data dictionary.

Transformer Truncation as Known Limitation

`truncation=True` on both HF pipelines — long reviews silently truncated to model max tokens. Documented in README rather than papered over. A known limitation stated is a known limitation managed.